applied sciences

MDPI

Article

An Easy to Use Deep Reinforcement Learning Library for AI Mobile Robots in Isaac Sim

Maximiliano Rojas *, Gabriel Hermosilla, Daniel Yunge and Gonzalo Farias

Escuela de Ingeniería Eléctrica, Pontificia Universidad Católica de Valparaíso, Av. Brasil 2147, Valparaíso 2362804, Chile
* Correspondence: maximiliano.rojas.l@mail.pucv.cl

Abstract: The use of mobile robots for personal and industrial uses is becoming popular. Currently, many robot simulators with high-graphical capabilities can be used by engineering to develop and test these robots such as Isaac Sim. However, using that simulator to train mobile robots with the deep reinforcement learning paradigm can be very difficult and time-consuming if one wants to develop a custom experiment, requiring an understanding of several libraries and APIs to use them together correctly. The proposed work aims to create a library that conceals configuration problems in creating robots, environments, and training scenarios, reducing the time dedicated to code. Every developed method is equivalent to sixty-five lines of code at maximum and five at minimum. That brings time saving in simulated experiments and data collection, thus reducing the time to produce and test viable algorithms for robots in the industry or academy.

Keywords: Isaac Sim; mobile robot; deep reinforcement learning; navigation

Citation: Rojas, M.; Hermosilla, G.; Yunge, D.; Farias, G. An Easy to Use Deep Reinforcement Learning Library for AI Mobile Robots in Isaac Sim. *Appl. Sci.* **2022**, *12*, 8429. https://doi.org/10.3390/app12178429

Academic Editors: Albert Smalcerz, Wojciech Kaczmarek and Jarosław Panasiuk

Received: 11 July 2022
Accepted: 19 August 2022
Published: 24 August 2022

1. Introduction

The fields of robotics and deep learning algorithms are gradually expanding to every field of human activity such as medicine, agriculture, economy, and transport, among others. Nowadays, mobile robots are increasingly crucial in the logistic and manufacturing industry [1] due to their ability to transport materials and packages quickly, efficiently, intelligently, and in a scalable way. However, not only that, there is increasing interest and development in assistive robots [2] that can take care of a limited range of tasks to make life easier for elders as personal use or transport medicament in a hospital.

Mobile robots move around, so it is imperative to find a way to test the algorithms before the end-user uses them. However, that can be time-consuming and expensive, mainly when deep reinforcement learning models behaviors. Here is when simulators come in handy. There are several general-purpose robot simulators such as MuJoCo [3], CoppeliaSim [4], PyBullet [5], and Gazebo [6]. All of them have tools that individuals are capable of using to add important features and possibilities to train and test algorithms in a safe and fast manner. For example, they can run tests in the cloud [7,8] and generate semi or photorealistic environments [9,10]. In addition, there is an open-source machine learning framework that uses deep learning through Tensorflow or Pytorch to train agents with deep reinforcement learning [11,12], among others.

Isaac Sim is NVIDIA's newest scalable robotics simulator capable of synthetic photorealistic data generation in physically accurate environments to develop, test, and manage robots based on artificial intelligence techniques [13,14]. Its primary purpose is to bring a realistic GPU-enabled physics to address the most common robotics problems such as manipulation, navigation, synthetic training data generation with a modular design, and a sim2real experiment [13,15]. Thus, it can be said that this simulator gathers the essential features of the ones mentioned earlier, making it a state-of-the-art tool to train robots under deep learning techniques. In this aspect, some libraries intend to, in one way or another, make it easier to test, develop, or train mobile robots. For example, Hall et al. [16] propose

BEAR (BenchBot environments for active robotics), which is a physics and visually realistic set of scenes with an API to foster research of robot spacial understanding; Tsoi et al. [17] propose SEAN (Social Environment for Autonomous Navigation), a high visual fidelity, open-source, and extensible social navigation platform with tools for navigation algorithms validation; and Guaman et al. [18] propose a deep learning framework for vehicle and pedestrian detection in rural roads.

Deep Reinforcement Learning (DRL) is concerned with solving sequential decision-making problems, and its structure can be expressed as a system with two elements: an environment and an agent. The first one produces information about itself (*state*); meanwhile, the former observes the state and with that selects or generates an *action*. The environment changes because of that transitioning to the next state, returning a *state* and a metric to determine how good the action was (reward). The cycle of state–action–reward is defined as a *time step* or *simulation step* [19].

The action-producing process maps states into actions, denoted as *policy*, that change the environment, which returns information that is used to modify the *policy* to maximize the reward, which is a direct indicator of how well the goal is (or not) achieved [19].

To our knowledge, no library provides an easy-to-use framework that intends to train robot agents through DRL with multiple robots and environments. This work precisely aims to fuse different APIs belonging to Isaac Sim and Gym to create a customizable training environment for mobile robots, which carries secondary benefits such as reducing the time dedicated to code. Every developed method is equivalent to sixty-five lines of code at maximum and five at minimum. Therefore, it is possible to divide the training process into robot modeling, environment creation, and deep learning configuration. The simplification of code through the library applies to all of them, so if a researcher wants to develop and test a deep learning navigation method, the whole process is reduced to a few lines of code using the available resources. We include a case of the study section that shows the results of the training of the Jetbot robot using a single custom environment and its evaluation in other realistic scenes. In addition, it is possible to run the same model with other differential and holonomic robots (with minor settings in the action space).

2. Methodology

This section presents an overview of the different components of the library and how they work. In the Isaac Sim section, the core functioning of the simulator and how it can be used to create several methods for robotic control are explained. In the next section, OpenAI Gym presents how this library approaches the problem of the environment definition, structure, and interaction with external elements. Later, the deep reinforcement learning section introduces a mathematical approach to the DRL issue as a partially observable Markov decision process. Finally, section Stable Baselines 3 introduces the library, which contains numerous agents and features to train the robot.

2.1. Isaac Sim

Isaac Sim bases its functionality on extensions that elements can be classified as: main (core functionality and control), sensors (creation and interface), asset conversion (importing robot tools), robot (main robot classes), and others (such as debugging and motion generation programs) [20]. Every one of them takes part in the functionality of the simulator; thanks to that, it is possible to use the code inside as standard python libraries, which allow the final user the option of doing everything available in the GUI coding.

One of the most important extensions for reinforcement learning is Isaac Gym. It allows the vectorization of custom environments, which means the classic process in which the simulation steps, logic environment, and its essential information and calculations are performed by the CPU, and the rendering and neural network training (or evaluation) by the GPU. That changes all the processes to be carried out with an end-to-end approximation by the GPU. It is possible because all the data are vectorized in the GPU (tensor

representation), which speeds up the simulation and allows a parallel training approach if necessary [14].

When it is necessary to manipulate the pose of prims (objects in the virtual environment), there are four main libraries: CORE, USD, PhysX, and DYNAMIC CONTROL [21]. All of them work with Cartesian three-dimensional position representation (x, y, z in centimeters) and a four-dimensional representation for the orientation (quaternions in radians or degrees) [22]. Using them together makes it possible to create a general class for element manipulation and several functions to control, manipulate and modify robots' assets. In the CORE extension, the Articulation and Robot class are created for such purposes, and we can add functions of the other extensions, such as Range-Based Sensor, Isaac Sensor, Simulation Application, Universal Robot Description File (URDF) or MuJoCo File (MJCF). These importers provide the rest of the necessary features to create any specific type of robot class that can be created from scratch. Isaac Sim does this with the programs that control the included robots such as Jetbot, Kaya, and Franka [23]. This process can be viewed in Figure 1 where the "Isaac Sim" box represents the simulator as a whole. From that, the "Extension API categories" box extracts some of the extensions that make all the simulator's functionalities possible. In addition, all of them have libraries with classes, which is important because, from those represented in the "Prim manipulation libraries" and "Base classes for robot interaction", boxes can be created as a class for custom robots ("Specific robot class" box), which is the foundational concept of the proposed work.

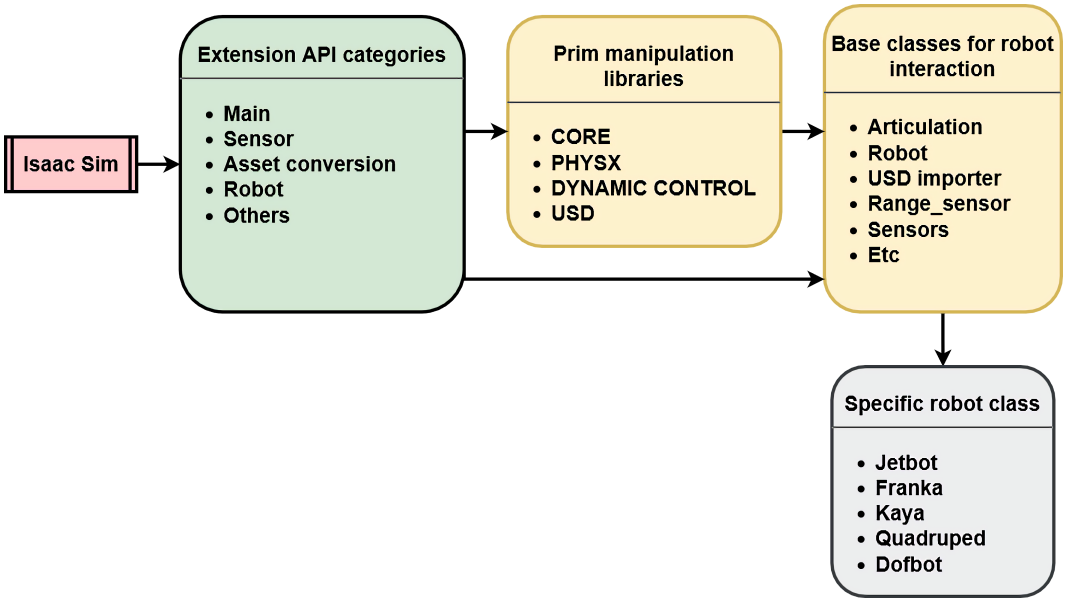

Figure 1. The internal process of Isaac Sim to control a specific robot.

2.2. OpenAI Gym

OpenAI Gym focuses on episodic training in the reinforcement learning paradigm, which means the agent's experience is processed in episodes, where the initial state of the learner is randomly sampled from a distribution. The interaction with the environment ends when a terminal state is reached. This API focuses on the environment's abstraction and how the agent interacts with it [24]. The Gym class also provides a way to specify information about the observation's data through the sub-class spaces.

An environment can be represented by the Gym class and must include the following elements [24]:

1. Declaration and Initialization: configuration of the metadata, render mode, frame rate, and several other initial configurations of the specific environment, for example, the maximum steps for episodes.

2. Construct observations from environment states: a method that translates the environment's states into observations.
3. Reset: a function that initializes a new episode, and it must return either an initial state's observation or a tuple of some auxiliary information.
4. Step: method that contains the main logic of the environment, processes actions, computes environment states and calculates rewards.
5. Close: a function that closes any resources used by the environment that are not always necessary.

With all these elements, it is possible to create a hybrid between a custom Gym environment and a general robot class compatible with Isaac Sim. The extensions provide the fundamental functions to manipulate any 3D robotic model, from joint manipulation to sensor creation. If the Gym structure is added, all these methods can be used as logic to manage and control the flow of an Isaac Sim environment. This idea is represented in Figure 2, where the "Isaac Sim" and "Gym" boxes represent the simulator and Gym API, respectively. The main elements that serve as construction blocks for the library's new methods can be extracted from those systems; these are represented in the "Easy to use Isaac Sim deep reinforcement learning library".

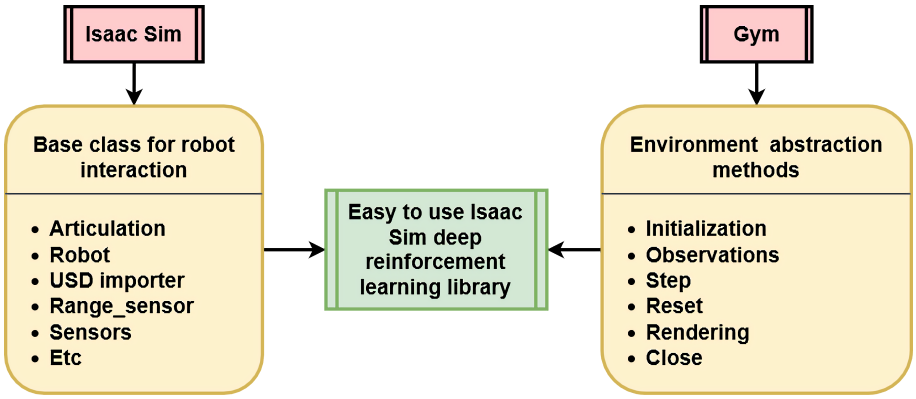

Figure 2. Combination of the different functionalities of Gym and Isaac Sim extensions to create a new library.

2.3. Deep Reinforcement Learning

The reinforcement learning problems essentially consist of an environment that generates information about itself, which is denoted as *states*, and an agent that makes sense of it to create some strategy or *policy* π to generate actions a to pursue or accomplish a specific task or *goal*.

In this regard, the simulator becomes a toolkit with several features, which allow the creation of virtual robots and environments that could be used for experimentation, being a tool for research and creating robust and easy-to-transfer results to reality.

The way an environment changes from one *state* s_t to the next one can be considered as a Markov decision problem (MDP); then, in every time step t, the next *state* s_{t+1} provides the probability distribution P conditioned by the entire history of the interactions *agent–environment*. That means the transition between s_t and s_{t+1} can be expressed as $P(s_{t+1}|(s_0, a_0), (s_1, a_1), \ldots, (s_t, a_t))$. In an MDP, the observable *state* always is the entire environment, but that rarely happens in reality; in this case, the MDP is described as a partially observable Markov decision process (POMDP) [19].

In MDP, the *agent* learns a function that produces or select actions, which means it learns a *policy* π, and that is the core functioning of reinforcement learning where the three primary mathematical expressions learned are:

- Strategy π, which translates *states* into *actions*: $a \sim \pi(s)$.
- Value function $V^{\pi}(s, a)$ or $Q^{\pi}(s, a)$ to estimate the expected return E of the reward R $E_{\tau}[R(\tau)]$.

- The environment's model $P(s'|(s,a))$.

Finally, the DRL approach is to learn the different available functions through deep neural networks, where the policy can be stochastic. That means choosing between a preset of actions given the esteemed reward of each one, denoted as $\pi(a|s)$, or generating the actions in each simulation step, denoted as $a \sim \pi(s)$ [19].

2.4. Stable Baselines 3

Given the library's structure, it is necessary to add an external one to use a DRL agent. Stable Baselines 3 (SB3) provides different agents: one algorithm (DQN) which estimates $V^{\pi}(s,a)$ and/or $Q^{\pi}(s,a)$ being the policy $\pi(a|s)$, and other five (PPO, SAC, TD3, DDPG, and A2C), which directly calculate the agent's actions with $a \sim \pi(s)$. In addition, the library provides the personalization and creation of custom environments, policies, and callbacks (a way of representing useful information for the user), which make it completely compatible with what is proposed in this work. PyTorch is used as a deep learning framework compatible with Isaac Sim [25].

Adding the Gym structure with the vectorization and parallelization capabilities of Isaac Gym makes it possible to create any physical and photorealistic custom environments for any agent of the SB3 DRL library. Figure 3 shows a general overview of how the library proposed works, including the agent's learning process.

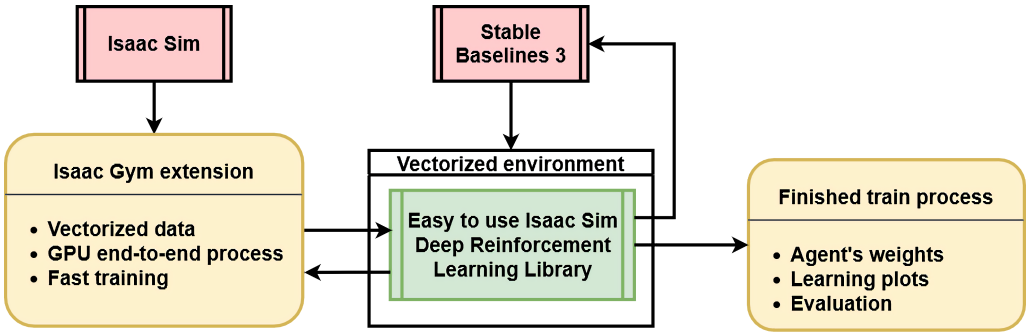

Figure 3. A general overview of how the library interacts with the Isaac Gym extension and Stable Baselines 3 to train an expert agent.

3. Library

The library can be downloaded from https://github.com/MaxiEIE/Easy_DRL_Isaac_Sim (accessed on 18 August 2022), where also the installation instructions can be found.

3.1. Structure

Below, in Figure 4, the general structure of the proposed library is presented. The union of three modules corresponding to Isaac Sim, Gym, and DRL stands out.

Three important files compose this library. The first one is isaac_envs.py with isaac_env class. Here are all the necessary methods to add, define, create and configure scenes and sensors. Many functions have an entry to specify the robot filled automatically with a global variable. Thus, every environment and sensor is configured without human intervention to fit the requirements of the selected robot. The second one is isaac_robots.py with the isaac_robot class. Here are all the necessary functions to import, control and acquire state data of a mobile robot. The third one is the env.py file that inherits the Gym class. Here, all the functions in the other classes are used following the general structure of the Gym API.

Isaac_envs class, when initialized, create the elemental world prims and configurations, such as running the physics engine, setting the rendering frequency, the units (in centimeters by default), and several initial configurations of the possible sensors and manipulation stage. Only one line of code is necessary to add an environment where two arguments are required: the scene and the robot's name (for custom environments only).

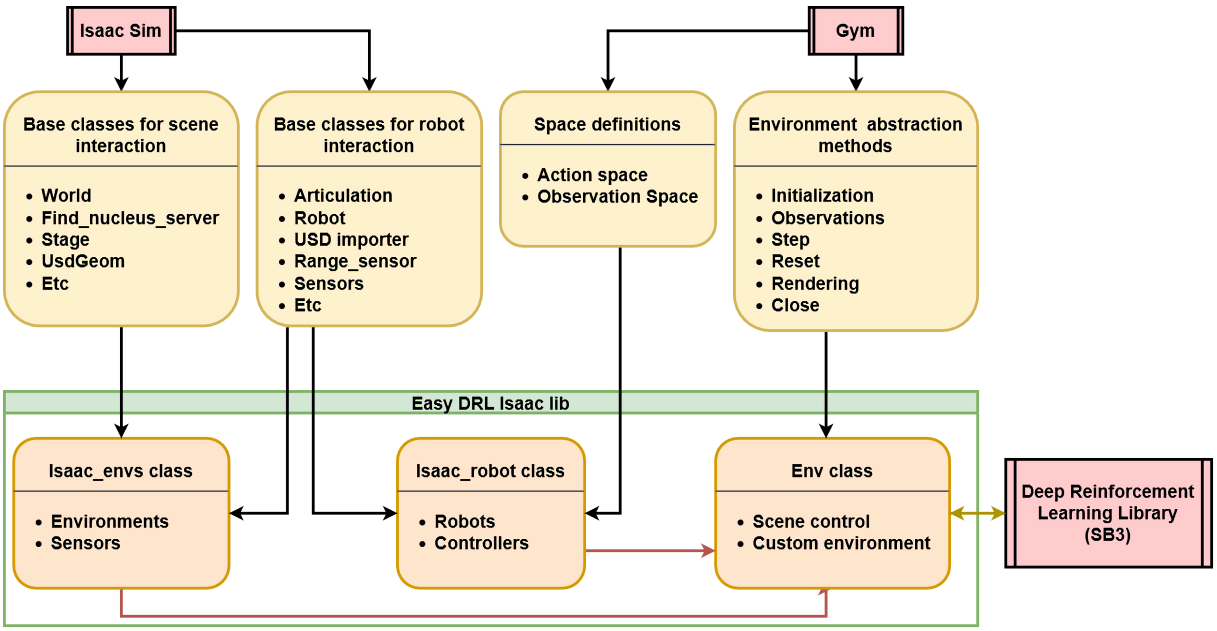

Figure 4. The main elements of the proposed library and how they interact with Isaac Sim, Gym, and SB3.

Isaac_robot class imports and configures the 3D robot asset to be used in the scene as an articulated prim. It mainly inherits from the Isaac Sim general class Robot, which in turn has the functions of the class Articulation, which implies every part of the robot is considered a joint. Thus, it can be measured and controlled. With these two elements, a way to manipulate the velocities for a differential and holonomic robot can be created.

Env class, which inherits from Gym class, establishes the structure to manipulate all the scene elements and the simulation's behavior. The other classes are imported and used here, the DRL environment parameters are set, actions take place, and the reward is calculated. Thus, to run training, this file must be used with the training's library SB3, and only three parameters are needed: the environment, robot, and sensors. In addition, a discrete or continuous action space is required as input. Table 1 summarizes the different classes' capabilities.

Table 1 resumes the capabilities available in the proposed library. The "isaac_robot class" column concentrates on a series of methods to acquire and set different particularities of the robot, such as its pose, wheel velocities in angular or linear form, controllers for the different types of a mobile base, and deep learning parameters related to training. The "isaac_envs class" column relates to the environment and sensor configuration. For the environment, it is possible to include the existing scenes in Isaac Sim or use a custom one that creates a map of random obstacles between the robot and the goal point. That can be further customizable for the number of obstacles and if a static map is necessary or not because, by default, it randomizes the spatial configuration of the elements when required (in the reset function of Gym, for example). Finally, the "env class" provides several methods to manage the scene, which serves as a training environment. For example, controlling the action space (discrete or continuous) and observation space for the different sensor inputs is possible. Here are the methods to customize the lidar and camera (RGB or RGB-D) parameters, such as the number of lasers or image size. Of course, the environments are called from this part of the library, enhancing the user experience and saving time in all these know-how tasks to train an expert agent.

All three main classes interact with other libraries to extract, condense, and simplify their characteristics. Later, these new functions handle a specific range of tasks expressed in the custom environment to configure and set up all the necessary assets. Dividing the

process into three allows adding other scenes and robots following the example within the files. This way, an easy method to train mobile robot agents with further user customization can be proposed.

Table 1. The capabilities of the three main elements of the library.

isaac_robot Class	isaac_envs Class	Env Class
Obtain wheels' linear velocities.	Set RGB or depth camera (in render or headless simulation).	Configure environment to be discrete or continuous.
Obtain wheels' linear velocities.	Set lidar with the required number of lasers (in render or headless simulation).	Different possible observations space.
Obtain lineal and angular velocity of the robot's chassis.	Obtain data from a camera or lidar.	Automatic configuration for available custom environment.
Obtain the distance to an object relative to the robot's chassis.	Generate random obstacle map.	Easy access to Issac Sim environments.
Obtain the angular difference between the front of the robot and the vector from the robot's base and a target.	Configure custom scenes.	Easy access to Issac Sim Sensors.
Obtain the robot's action space.		Easy access to Issac Sim robots.
Obtain the set of discrete actions for discrete environments.		
Set wheels' linear velocities.		
Set 3D robot position and quaternion orientation.		
Differential controller for two-wheeled robots.		
Holonomic controller for three-wheeled robots.		

3.2. Robots, Sensors, and Environments

This library comes with easy access and configuration to all environments of the Isaac Sim simulator [26,27], plus one custom-made for random obstacle generation. The supported mobile robots are differentials and three-wheeled holonomic ones. Some Isaac Sim mobile robots are included, automatically configured, and ready to use. In addition, several sensors are included to measure different robots and environment states. Table 2 presents the details of these elements.

Table 2. Library's available resources.

Environments (Scenes)	Robots	Sensors
Three different flat grids (normal, black and curved).	Jetbot (differential).	Wheel lineal velocity sensor (encoder).
A simple, tiny room with a table at the center.	Carter V1 (differential).	Robot's base lineal velocity sensor (3D velocity magnitude).
Four houses of different sizes and obstacles.	Transporter (differential).	Robot's base angular velocity (of the yaw angle).
One floor of a hospital building.	Kaya (holonomic).	Customizable RGB camera.
One floor of an office building.		Customizable depth camera.
A custom random obstacle map.		Customizable lidar (range sensor).

Custom robots and scenes can be easily added through URDF (for the robots) or USD (for both) files, and every sensor can be modified to some extent. Figure 5 shows some examples of the included assets.

3.3. Control Types

The isaac_robot class has two methods designed to control both types of mobile robots, but to generalize to any possible model, it is necessary to derive a set of equations that models the kinematics related to the wheel's velocities. The information necessary to

perform that task can be found in Figure 6A for a differential configuration and in Figure 6B for holonomic ones.

Figure 5. Some of the Isaac Sim's included assets. (**A**) Tiny room, (**B**) Office floor, (**C**) Carter V1 robot, (**D**) Kaya robot, (**E**) Warehouse, (**F**) Hospital floor, (**G**) Jetbot robot, (**H**) Smart Transport Robot.

Here, W is the angular base velocity, V is the lineal base velocity, W_L and W_R are the angular velocities of the left and right wheel; V_a, V_b and V_c are the linear velocities of the holonomic wheels, L is the distance from the center to the holonomic wheel, R is the distance between the differential wheels, and r is the wheel's radius. The equation that describes the wheel's velocities for a general differential robot is:

$$W_R = \frac{2V + WR}{2r} \tag{1}$$

$$W_L = \frac{2V - WR}{2r} \tag{2}$$

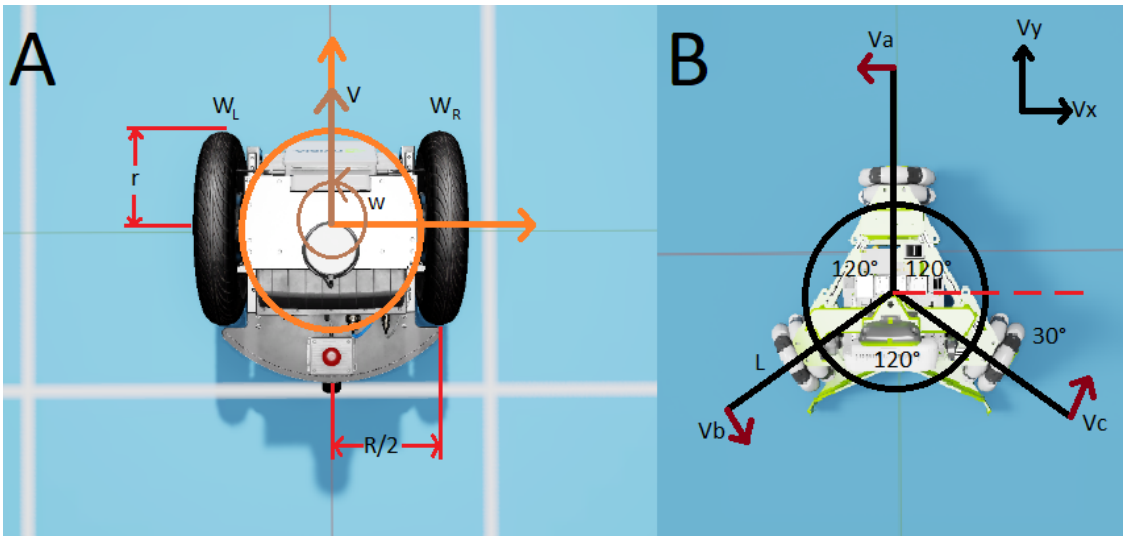

Figure 6. Geometric representation of the different robot bases included in the library, being (**A**) differential, and (**B**) holonomic.

For the case of a holonomic robot, the equations are:

$$V_A = -V_x + WL \tag{3}$$

$$V_B = V_x \cos(-30^\circ) + V_y \cos(-30^\circ) + WL = \frac{V_x}{2} - \frac{\sqrt{3}V_y}{2} + WL \tag{4}$$

$$V_C = V_x \cos(30^\circ) + V_y \sin(30^\circ) + WL = \frac{V_x}{2} + \frac{\sqrt{3}V_y}{2} + WL \tag{5}$$

Then, if $V = Wr$, a relationship between linear and angular velocities can be obtained:

$$W_A = \frac{-V_x + WL}{R} \tag{6}$$

$$W_B = \frac{V_x - \sqrt{3}V_y + 2WL}{2R} \tag{7}$$

$$W_C = \frac{V_x + \sqrt{3}V_y + 2WL}{2R} \tag{8}$$

3.4. Deep Reinforcement Learning Configurations

Some essential features of this library for DRL is that the Env class can be used as a discrete or continuous gym environment changing only one line of code. Every robot has its own set of discrete actions and continuous action space that can be obtained by calling the proper method in the isaac_robot class. We include an example where the observation space is divided into different modes: robot state information, range sensor (lidar of 12 points by default), camera RGB (size of $3 \times 128 \times 128$ pixels by default), depth camera (size of $1 \times 128 \times 128$ pixels by default), and information related to the target relative to the robot position (such as distance and angular difference regardless orientation).

Another important aspect is the reward function per step. In this case, we define it as follows:

$$r = \begin{cases} -d_t\left(1 - \frac{step_i}{step_{max}}\right) & ;\text{if goal isn't achieved} \\ -p & ;\text{if robot collides with the obstacle} \\ r_l\left(1 - \frac{step_i}{step_{max}}\right) & ;\text{if robot achieves goal} \end{cases} \tag{9}$$

where r is the reward, d_t is the distance between the robot and the target point, $step_i$ is the i-th episode step, $step_{max}$ is the maximum steps per episode, p is a penalization if the robot collides with an obstacle, and r_l is a maximum reward value if the agent accomplishes the goal. The different reward values, from top to bottom, of Equation (9) motivate different behaviors, which are:

1. When the robot is still trying to position itself on the target, the reward function is the negative distance between them d_t weighted by a factor $\left(1 - \frac{step_i}{step_{max}}\right)$ that decreases its value when more steps are executed $step_i$. The idea with this is that it motivates a fast reduction of the related distance.
2. The second element comes in action when the robot approaches too close to an obstacle, so a fixed penalization p is set as a reward.
3. The last possible reward is when the robot accomplishes the goal; thus, a positive landing reward r_l weighted by the total steps $step_i$ of the episode is provided. The more steps, the less landing reward.

4. Case of Study

A case study is presented to summarize and explain how the library's components work and interact with the external ones. The general configuration of the experiment is a differential robot Jetbot as the agent, which includes a custom scene as the environment and the reward function (Equation (9)). Figure 7 illustrates the entire process.

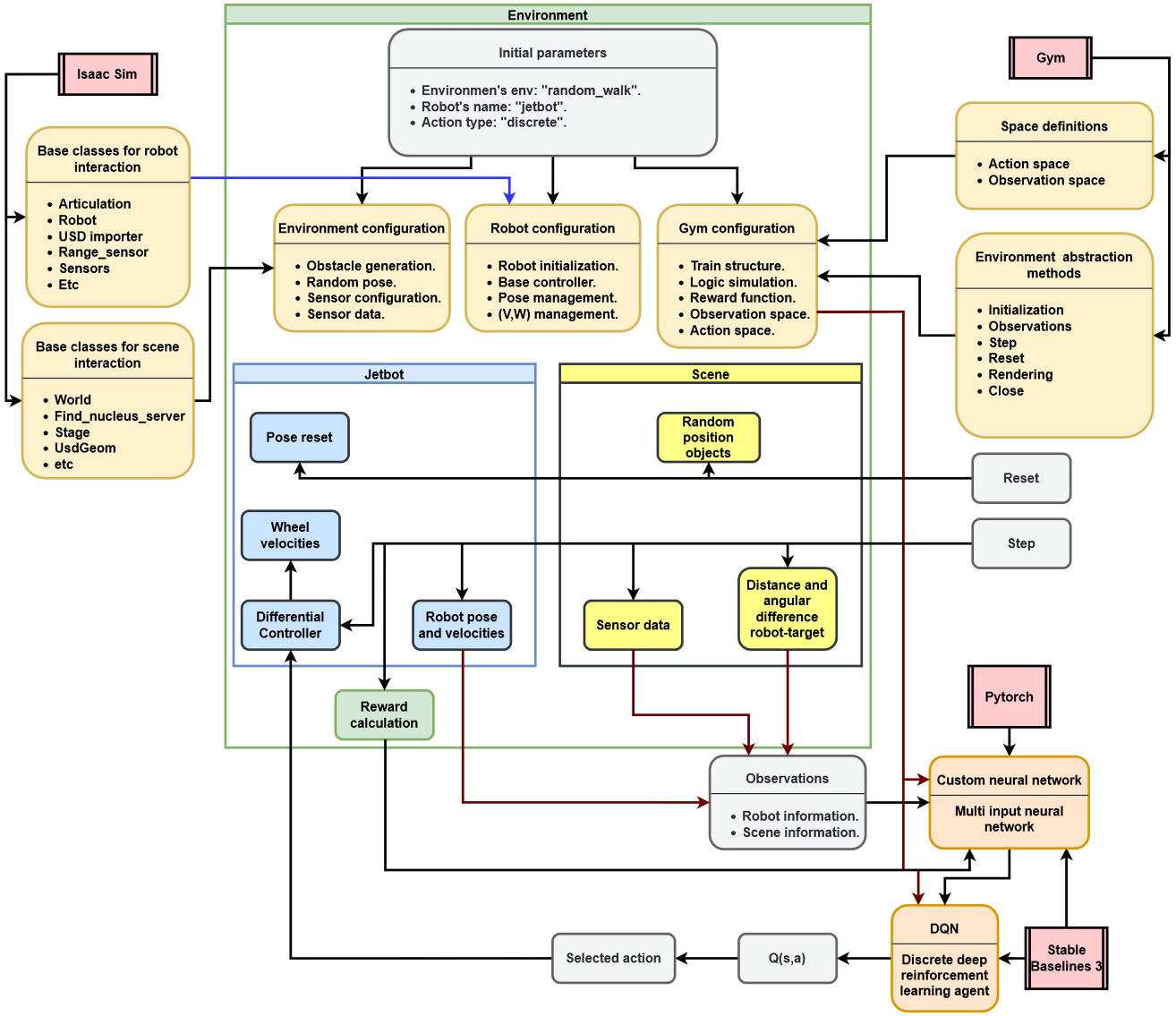

Figure 7. Isaac Sim's study case structure, logic, and information flow.

The following elements must be specified to use the three main classes: the robot, the environment, and the action type; with that, all the configurations of the necessary elements needed for training are ready to use. Several elements have default parameters but are changeable if required. With that, the scene (in this case, "random_walk") and the robot (Jetbot) are created. At the beginning of the training process, the reset function of Gym runs, so the robot's pose is randomized, and a new obstacle map in the scene is generated (a custom fixed obstacle map is available, too). Later in the step function, all the logic and environment behavior are executed through the corresponding functions, particularly the wheel's velocities of the robot, where the DQN agent from SB3 esteems the $Q(s, a)$ values for each set of possible actions, and one is selected. The differential controller traduces the angular and linear velocity of the robot's base to the wheels. All the relevant information comes from the robot and environment class. It is used as input to the custom neural network (created with Pytorch), which, with the reward, changes the action evaluation of the DQN agent at each step.

The primary way Isaac Sim extensions and the Gym library interact with the proposed work is through the configuration of the scene, robot, and all the methods and functions that make it easier to train the agent. In this study case, Jetbot learns how to reach a target point without colliding with obstacles with a DQN agent. Its hyperparameters are in Table 3.

Table 3. Hyperparameters of the DQN agent.

Parameter	Value
Max steps per training	3,000,000 [steps]
Max steps per episode	3000 [steps]
Buffer size	800,000 [steps]
Learning rate	0.00015 [-]
Exploration factor	0.35 [-]

Figure 8 shows the graphics of the training. The average reward increases its value significantly between the steps 800k to 1400k, where the policy finds its local optimum. That alone is insufficient to demonstrate that the robot learns how to navigate to the target point. That is why, as a complement, the average episode length is presented. The decrease in value makes it clear that the robot learns how to accomplish the goal. In addition, an evaluation of 30 episodes was made to extract some useful information about the quality of the learned policy π; the result presented in Table 4. In Figure 9, we present some robot trajectories as an example of the learned behavior.

Table 4. DQN agent evaluation metrics.

Parameter	Value
Rate of success	86.7%
Episode's time	27.6 [s]
Episode's steps	1619 [steps]
Robot trajectory	504.1 [cm]

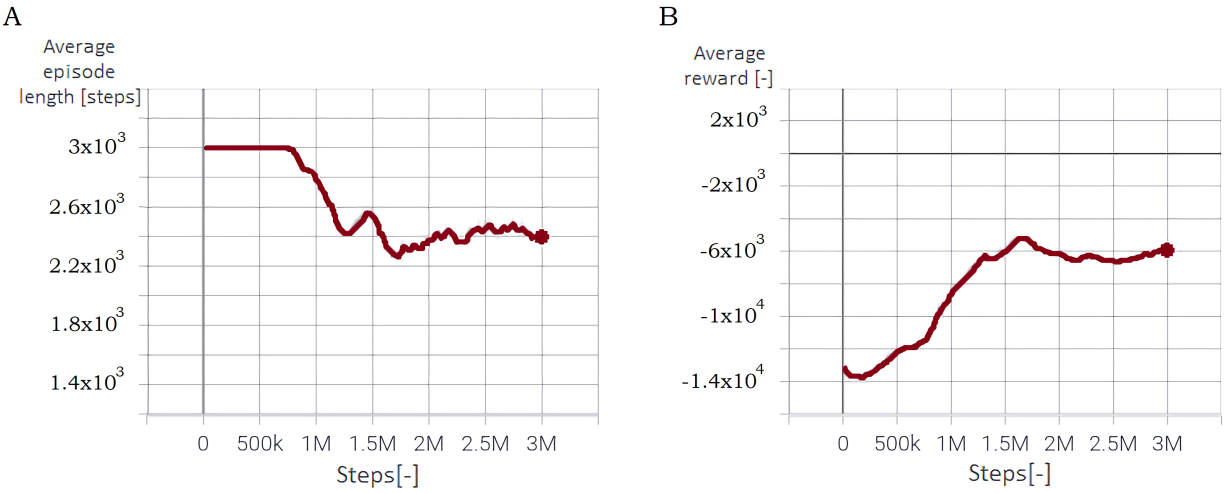

Figure 8. Function of the study case: (**A**) Average episode length, (**B**) Average reward..

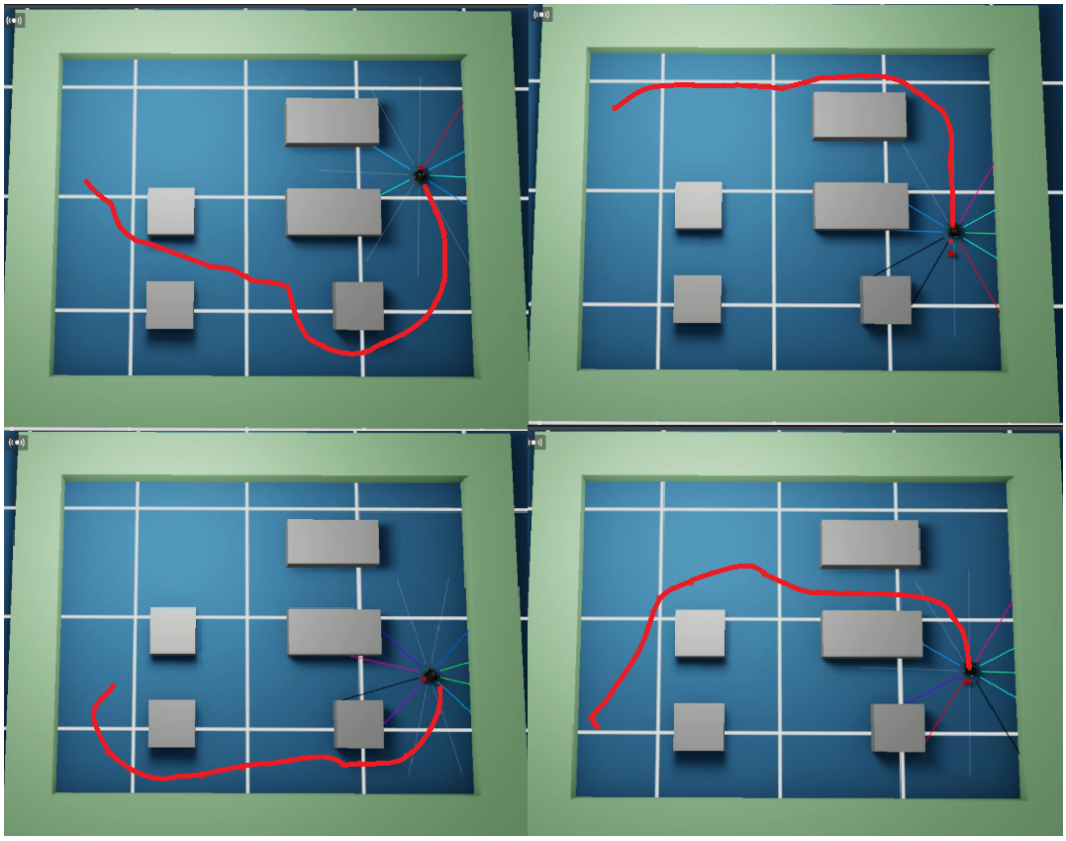

Figure 9. Agent's learned trajectories in Isaac Sim simulator with a custom environment.

5. Discussion

The study's case reveals and explains how to train an expert agent successfully. The robot learns how to move efficiently in the test scene to reach the target point using a lidar configured to have only eight points of measure with a maximum distance of 1 meter for measurement and obstacle avoidance. No camera is used, but the state of the robot and the valuable information about the distance and angle difference between the robot and the target can be quickly obtained through the existing methods created with the Isaac Sim extensions. The manipulation of the wheel's velocities can be expressed directly or through the needed chassis velocities. In the last case, the differential controller transforms them directly to manipulate the joints of the wheels for any included robot.

The three main classes generate and configure everything needed to run the simulation. For example, isaac_env has several environments and sensors ready to use and configure if needed. isaac_robot has several Isaac Sim robots, importing them directly from the USD files. That allows any custom robot following the Isaac Sim way to import them. Finally, the env_class concentrates all the OpenAI Gym environment structure and abstraction to manipulate all the elements according to the already written scene and training logic. All of these classes are made from the basic methods of the corresponding libraries. That is significant because it allows any research to change, modify and expand whatever is necessary to make this work more fitted to particular cases.

The results can be further improved with an adequate reward function. This case is only an example of what the proposed library can do, showing that it can generate and test new policies for mobile robots.

6. Conclusions

This paper presents a new library for experiments and DRL training with advanced mobile robots in physic and photorealistic environments. The library uses several standard

libraries such as OpenAI Gym, Pytorch, Isaac Sim extensions, and SB3 to create more general and usable methods to perform experiments and create expert agents.

It is easy to use, configure, and customize all different robots, sensors, environments, and methods that allow and facilitate research in AI-based mobile robots using Isaac Sim as the simulator, concealing configuration problems to training expert agents. Furthermore, the work adds new possibilities regarding the existing technology for programming in realistic virtual environments and scenarios, which saves valuable time that can be used in virtual experiments, and data collection, thus reducing the time to produce a viable algorithm for the industry and the academy.

Future work involves expanding the available custom environments for virtual training with domain randomization of its elements and scene configuration. That is very important because it allows the agents to learn more rich information and better feature extraction from the autonomous randomness of the scene and environment start variables, such as the target position value, which results in better policies. Another area of forthcoming development is fully implemented data vectorization for massive parallel training in headless mode. Thus, it is possible to generate shorter training periods at the expense of increasing the usage of computational resources, for the first approximation must be with new and simpler custom environments for single GPUs. Finally, for sim2real experiments, the implementation of ROS2 would be beneficial. Consequently, the data transmission between the different sensors, microcontrollers, and computers could be standardized, making the DRL algorithms easiest to implement and test in the real world.

Author Contributions: Investigation, M.R.; Methodology, G.H.; Supervision, D.Y.; Validation, G.F.; All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported in part by FONDECYT under Grant 1191188.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: Not applicable.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Fragapane, G.; de Koster, R.; Sgarbossa, F.; Strandhagen, J.O. Planning and control of autonomous mobile robots for intralogistics: Literature review and research agenda. *Eur. J. Oper. Res.* **2021**, *292*, 405–426. [CrossRef]
2. Gonzalez-Aguirre, J.A.; Osorio-Oliveros, R.; Rodríguez-Hernández, K.L.; Lizárraga-Iturralde, J.; Morales Menendez, R.; Ramírez-Mendoza, R.A.; Ramírez-Moreno, M.A.; Lozoya-Santos, J.d.J. Service Robots: Trends and Technology. *Appl. Sci.* **2021**, *11*, 10702. [CrossRef]
3. Todorov, E.; Erez, T.; Yuval, T. MuJoCo: A physics engine for model-based control. *IEEE/RSJ Int. Conf. Intell. Robot. Syst.* **2012**, *1*, 5026–5033
4. Rooban, S.; Suraj, S.D.; Vali, S.B.; Dhanush, N. CoppeliaSim: Adaptable modular robot and its different locomotions simulation framework. *Mater. Today Proc.* **2008**, *10*, 142–149. [CrossRef]
5. Bullet Real-Time Physics Simulatio. Available online: https://pybullet.org/ (accessed on 21 June 2022).
6. Gazebosim: Simulate before You Build. Available online: https://gazebosim.org/home (accessed on 21 June 2022).
7. Liu, Z.; Liu, W.; Qin, Y.; Xiang, F.; Gou, M.; Xin, S.; Roa, M.; Calli, B.; Su, H.; Sun, Y.; et al. OCRTOC: A Cloud-Based Competition and Benchmark for Robotic Grasping and Manipulation. *IEEE Robot. Autom. Lett.* **2021**, *10*, 486–493. [CrossRef]
8. PyBullet in a Colab. Available online: https://pybullet.org/wordpress/index.php/2021/04/15/pybullet-in-a-colab/ (accessed on 21 June 2022).
9. Morrical, N.; Tremblay, J.; Lin, Y.; Tyree, S.; Birchfield, S.; Pascucci, V.; Wald, I. NViSII: A Scriptable Tool for Photorealistic Image Generation. *arXiv* **2021**. [CrossRef]
10. Greff, K.; Belletti, F.; Beyer, L.; Doersch, C.; Du, Y.; Duckworth, D.; Fleet, D.; Gnanapragasam, D.; Golemo, F.; Herrmann, c.; et al. Kubric: A Scalable Dataset Generator. In Proceedings of the CVPR, New Orleans, LA, USA, 19–24 June 2022; pp. 3749–3761.
11. Wang, C.; Zhang, Q.; Tian, Q.; Li, S.; Wang, X.; Lane, D.; Petillot, Y.; Wang, S. Learning Mobile Manipulation through Deep Reinforcement Learning. *Sensors* **2020**, *20*, 939. [CrossRef] [PubMed]
12. Yang, X.; Ze, J.; Wu, J.; Lai, Y. An Open-Source Multi-goal Reinforcement Learning Environment for Robotic Manipulation with Pybullet. *TAROS* **2021**, *22* 14–24._2. [CrossRef]

13. NVIDIA Isaac Sim. Available online: https://developer.nvidia.com/isaac-sim (accessed on 21 June 2022).
14. Makoviychuk, V.; Wawrzyniak, L.; Guo Y.; Lu, M.; Storeym, K.; Macklin, M.; Hoeller, D.; Rudin, N.; Allshire, A.; Handa, A.; et al. Isaac Gym: High Performance GPU-Based Physics Simulation For Robot Learning. *arXiv* **2021**. [CrossRef]
15. Figueredo, F.; Buarque, A.; Natário, J.; Teichrieb, V. Simulating real robots in virtual environments using NVIDIA's Isaac SDK. *SVR* **2019**, *1*, 47–48. [CrossRef]
16. Hall, D.; Talbot, D.; Bista, S.; Zhang, H.; Smith, R.; Dayoub, F.; Sünderhauf, N. BenchBot environments for active robotics (BEAR): Simulated data for active scene understanding research. *Int. J. Robot. Res.* **2022**, *41*, 259–269. [CrossRef]
17. Tsoi, N.; Hussein, M.; Espinoza, J.; Ruiz, X.; Vázquez, M. SEAN: Social Environment for Autonomous Navigation. *arXiv* **2020**. [CrossRef]
18. Barba-Guaman, L.; Eugenio Naranjo, J.; Ortiz, A. Deep Learning Framework for Vehicle and Pedestrian Detection in Rural Roads on an Embedded GPU. *Electronics* **2020**, *9*, 589. [CrossRef]
19. Loon Keng, W.; Graesser, L. *Foundation of Deep Reinforcement Learning Theory and Practice in Python*, 2nd ed.; Addison-Wesley Professional: Boston, MA, USA, 2020.
20. Isaac Sim: Extensions API. Available online: https://docs.omniverse.nvidia.com/py/isaacsim/index.html (accessed on 26 June 2022).
21. Dynamic Control. Available online: https://docs.omniverse.nvidia.com/app_isaacsim/app_isaacsim/ext_omni_isaac_dynamic_control.html (accessed on 26 June 2022).
22. Conventions Reference. Available online: https://docs.omniverse.nvidia.com/app_isaacsim/app_isaacsim/reference_conventions.html (accessed on 26 June 2022).
23. Overview and Fundamentals. Available online: https://docs.omniverse.nvidia.com/app_isaacsim/app_isaacsim/tutorial_cortex_overview.html (accessed on 26 June 2022).
24. Brockman, G.; Cheung, V.; Pettersson, L.; Schneider, J.; Schulman, J.; Tang, J.; Zaremba, W. OpenAI Gym. *arXiv* **2016**. [CrossRef]
25. Stable Baselines 3. Available online: https://www.ai4europe.eu/sites/default/files/2021-06/README_5.pdf (accessed on 26 June 2022).
26. Included Environments and Robots. Available online: https://docs.omniverse.nvidia.com/app_isaacsim/app_isaacsim/reference_assets.html (accessed on 26 June 2022).
27. Isaac Sensor. Available online: https://docs.omniverse.nvidia.com/app_isaacsim/app_isaacsim/ext_omni_isaac_isaac_sensor.html (accessed on 26 June 2022).